

2209/A

ÜNİVERSİTE ÖĞRENCİLERİ ARAŞTIRMA PROJELERİ DESTEK
PROGRAMI

SONUÇ RAPORU

PROJE BAŞLIĞI: **8-Bit Bilgisayar Mimarisi ve Komut Seti
Tasarımı ve Gerçekleştirilmesi**

PROJE YÜRÜTÜCÜSÜNÜN ADI: **Musa Akyüz**

DANIŞMANININ ADI: **Dr. Öğr. Üyesi Ümit Şentürk**

GENEL BİLGİLER

PROJENİN KONUSU	8 bitlik işlemci mimarisi tasarlanması
PROJE YÜRÜTÜCÜSÜNÜN ADI	Musa Akyüz
DANIŞMANIN ADI	Dr. Öğr. Üyesi Ümit Şentürk
PROJE BAŞLANGIÇ VE BİTİŞ TARİHLERİ	01.04.2022 / 20.09.2024

1. Giriş

Günümüzde, birçok işlemci üreten firma ve bu firmaların arasındaki rekabete baktığımızda hepsinin daha küçük alanda daha hızlı işler yaptıran elektronik devreler üretmeye çalıştığını görüyoruz. Bu çalışmada, nanometreler düzeyinde yapılan yarışın, büyük hale dönüştürülmüş versiyonu tanıtılmaktadır. Bu çalışma, işlemci içerisindeki mantıksal ve aritmetik hesaplama, kaydetme, giriş-çıkış ve kaydırma birimlerinin basit elektronik elemanlar ve tümleşik devreler ile tasarımını ve birimlerin birbirleriyle senkronize şekilde haberleşmesini içermektedir. Tasarlanan komut seti sayesinde bu birimlerin bit düzeyindeki işlemleri kontrol edilmektedir. 8 bit veri yollu tek vuruşluk işlemci tasarlanmıştır. Hafıza birimi işlemci içerisinde konumlandırılmıştır. Bu sayede günümüz işlemcilerinin bir dezavantajı olan hafıza biriminden veri işleme süresi ve gecikmesinin önüne geçilmeye çalışılmıştır.

2. Rapor dönemlerinde yapılan çalışmalar

2.1 Komut Seti Tasarlanması / Düzenlenmesi

Komut seti, işlemci üzerindeki kontrol elemanlarının yönetilmesini, kontrol edilmesini sağlayan ve işlemciye o anda hangi işlemi yapacağını söyleyen sistemdir. Bir başka deyişle bir komut çalıştırma esnasında işlemci üzerinde bulunan birçok alanın, gelen komutlara göre cevap verip vermeyeceğini, ortak veri yoluna (BUS) erişip erişmeyeceğini, o esnada hangi veri yollarının (BUS) kullanılacağını hangilerinin kullanılmayacağını belirleyebilmemiz için her komut için benzersiz bir ifade içeren tablodur. Bu sayede bu dokümanda ileride anlatacağımız Kontrol Ünitesi, gelen komutlara göre işlemcinin belli bölgelerini kontrol eder.

Projenin öneri formunda belirtmiş olduğumuz komut setinde ufak değişiklikler yapıldı. Gereksiz görülen komutlar kaldırılarak daha az komut ile temiz bir komut seti tasarlandı.

Proje Öneri Formundaki			Son hali			Değişiklik
	Komut	Açıklama		Komut	Açıklama	
1	ADD	İki kaydedicideki veriyi toplar	1	ADD	İki kaydedicideki veriyi toplar	-
2	ADDIMM	Bir kaydedicideki veri ile anlık değeri toplar	2	ADDIMM	İlk kaydedicideki veri ile anlık değeri toplar	-
3	SUB	İki kaydedicideki veriyi çıkarır	3	SUB	İlk kaydediciden ikinci kaydediciyi çıkarır.	-
4	SUBIMM	Bir kaydedicideki veriden anlık değeri çıkarır	4	SUBIMM	İlk kaydedicideki veriden anlık değeri çıkarır	-
5	AND	İki kaydedicideki verileri VE işlemine sokar	5	AND	İki kaydedicideki verileri VE işlemine sokar	-
6	ANDIMM	Bir kaydedicideki veri ile anlık değeri VE işlemine sokar	6	ANDIMM	İlk kaydedicideki veri ile anlık değeri VE işlemine sokar	-
7	OR	İki kaydedicideki verileri VEYA işlemine sokar	7	OR	İki kaydedicideki verileri VEYA işlemine sokar	-
8	ORIMM	Bir kaydedicideki veri ile anlık değeri VEYA işlemine sokar	8	ORIMM	İki kaydedicideki veri ile anlık değeri VEYA işlemine sokar	-

9	SHIFLEFT	Bir kaydedicideki veriyi bir BİT sola kaydırır	9	SHIFLEFT	İlk kaydedicideki veriyi bir BİT sola kaydırır	-
10	SHIFRIGHT	Bir kaydedicideki veriyi bir BİT sağa kaydırır	10	SHIFRIGHT	İlk kaydedicideki veriyi bir BİT sağa kaydırır	-
11	LBFROMIMM	Anlık değerden aldığı veriyi hedef kaydediciye yükler	11	IMMTOREG	Anlık değerden aldığı veriyi hedef kaydediciye yükler	Adı Değiştirildi
12	LBFROMMEM	Hafızadan aldığı veriyi hedef kaydediciye yükler, Hafıza adreslerken anlık değeri kullanır	12	MEMTOREG	Hafızadan aldığı veriyi hedef kaydediciye yükler, Hafıza adreslerken anlık değeri kullanır	Adı Değiştirildi
13	REGTOREG	Bir kaydedicideki veriyi başka bir kaydediciye aktarır, yükler	13	REGTOREG	İkinci kaydedicideki veriyi hedef kaydediciye aktarır	-
14	SBFROMIMM	Bir kaydedicideki veriyi adres olarak kullanır ve hafıza birimine anlık değeri yazar				Kaldırıldı
15	SBFROMREG	Anlık değerdeki veriyi adres olarak kullanır ve hafıza birimine kaydedicideki veriyi yazar	14	REGTOMEM	Anlık değerdeki veriyi adres olarak kullanır ve hafıza birimine ilk kaydedicideki veriyi yazar	Adı Değiştirildi
16	BEQ	İki kaydedicideki verinin eşit olma durumunu karşılaştırır. Eğer eşitse anlık değer kadar komut atlar	15	BEQ	İki kaydedicideki verinin eşit olma durumunu karşılaştırır. Eğer eşitse anlık değer kadar komut atlar	-
17	BNE	İki kaydedicideki verinin eşit olmama durumunu karşılaştırır. Eşit değilse anlık değer kadar komut atlar	16	BNE	İki kaydedicideki verinin eşit olmama durumunu karşılaştırır. Eşit değilse anlık değer kadar komut atlar	-
18	BLT	Birinci kaydedicideki verinin ikinci kaydedicideki veriden küçük olma durumunu karşılaştırır. Eğer büyükse, anlık değer kadar komut atlar	17	BLT	Birinci kaydedicideki verinin ikinci kaydedicideki veriden küçük olma durumunu karşılaştırır. Eğer büyükse, anlık değer kadar komut atlar	-
19	BGT	Birinci kaydedicideki verinin ikinci kaydedicideki veriden büyük olma durumunu karşılaştırır. Eğer büyükse, anlık değer kadar komut atlar	18	BGT	Birinci kaydedicideki verinin ikinci kaydedicideki veriden büyük olma durumunu karşılaştırır. Eğer büyükse, anlık değer kadar komut atlar	-
20	BGE	Birinci kaydedicideki verinin ikinci kaydedicideki veriden büyük ve eşit olma durumunu karşılaştırır. Eğer büyük ve eşitse, anlık değer kadar komut atlar	19	BGE	Birinci kaydedicideki verinin ikinci kaydedicideki veriden büyük ve eşit olma durumunu karşılaştırır. Eğer büyük ve eşitse, anlık değer kadar komut atlar	-
21	BLE	Birinci kaydedicideki verinin ikinci kaydedicideki veriden küçük ve eşit olma durumunu karşılaştırır. Eğer	20	BLE	Birinci kaydedicideki verinin ikinci kaydedicideki veriden küçük ve eşit olma durumunu karşılaştırır. Eğer küçük ve eşit ise,	-

		küçük ve eşit ise, anlık değer kadar komut atlar			anlık değer kadar komut atlar	
22	PRINTREG	Bir kaydedicideki veriyi çıkış birimine gönderir	21	PRINTREG	İlk kaydedicideki veriyi çıkış birimine gönderir	-
23	PRINTREGANDLOAD	Bir kaydedicideki veriyi çıkış birimine gönderir ve başka bir kaydediciye yükler				Kaldırıldı
24	PRINTMEMFROMREG	Bir kaydedicideki veriyi adres olarak kullanır ve hafıza birimine gönderir. O adresteki veriyi çıkış birimine gönderir				Kaldırıldı
25	PRINTMEMFROMIMM	Anlık değeri adres olarak kullanır ve hafıza birimine gönderir. O adresteki veriyi çıkış birimine gönderir	22	IMMTOPRINTMEM	Anlık değeri adres olarak kullanır ve hafıza birimine gönderir. O adresteki veriyi çıkış birimine gönderir	Adı Değiştirildi
26	WRITETOREG	Giriş biriminden gelen veriyi bir kaydediciye yükler	23	WRITEREG	Giriş biriminden gelen veriyi bir kaydediciye yükler	Adı Değiştirildi
27	WRITETOMEMFROMREG	Bir kaydedicideki veriyi adres olarak kullanır ve giriş biriminden gelen veriyi hafıza birimine yükler				Kaldırıldı
28	WRITETOMEMFROMIMM	Anlık değerden aldığı veriyi adres olarak kullanır ve giriş biriminden aldığı veriyi hafıza birimine yükler	24	IMMTOWRITEMEM	Anlık değerden aldığı veriyi adres olarak kullanır ve giriş biriminden aldığı veriyi hafıza birimine yükler	Adı Değiştirildi
			25	CLEARALLREG	Tüm kaydedicilerin verilerini sıfırlar	Eklendi
			26	CLEARSPECREG	İlgili kaydedicinin verisini siler	Eklendi

Tablo 1

Önceki ve sonraki komut seti karşılaştırması ve değişiklik raporu

Tablo 1’den de görüleceği üzere proje önerisinde sunulan 28 komuttan 4 ü kaldırılıp 2 yeni komut eklenmiştir. 26 komuta düşürülmüştür.

Komutları incelediğimizde, işlemcinin toplama ve çıkarma (aritmetik) işlemlerini, VE (AND) ile birleştirme ve VEYA (OR) ile birleştirme (mantık) işlemlerini destekleyeceğini görüyoruz. Bunlar bu yazıda ileride anlatılacak olan Aritmetik Mantık Birimi (Arithmetic Logic Unit - ALU) yazısında değinilecektir. Aynı zamanda sola ve sağa kaydırma işlemleri yapılabilir. Bu da projenin geleceğinde çarpma ve bölme işlemleri ekleneceği zaman kullanılabilecektir. Karşılaştırma işlemleri için 6 farklı karşılaştırma türüne olanak tanıyan komutlar yerleştirilmiştir. Bunun sayesinde dallanma komutları işlenebilecek ve programların olmazsa olmazı olan eğer (if - else) ve döngüler (for - while) ifadeleri ile programlama yapılabilecektir. Ayrıca işlemci bünyesinde BELLEK (RAM) bulundurduğu için bellek komutları da komut setinde yer almaktadır. Giriş

Çıkış birimi sayesinde dışarıdan bir giriş (input) alınabilecek veya işlemci içerisindeki bir veri çıkış (output) yapılabilecektir.

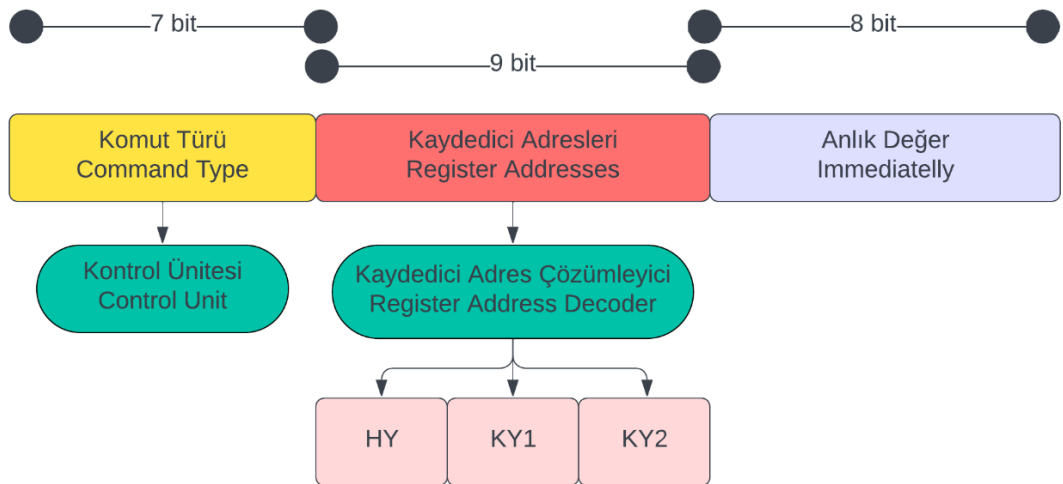
2.2 Program Belleği Tasarımı ve Komut İşleme Mantığı (Command Handling)

Program belleği adından da anlaşılacağı üzere bir bellektir ve içerisinde işlemcinin art arda işleyeceği komutları barındırır. Komutlar farklı uzunlukta ve türde olabilirler. Komut seti tasarlanırken birçok kez incelediğim RISC-V instruction Set buna örnektir. Ancak bu projede tüm komutlar aynı ve 24 bit uzunluğundadır.

Şekil 1'den de görüleceği üzere ilk 7 bit komut türünü belirtmektedir. 2^7 adet komut türü belirtilebilir. Tablo 1'de sadece 24 adet komut tasarlanmıştır. Projenin geleceğinde yeni ihtiyaçlar göre komutlar türleri oluşturulabilir. Komut türü bilgisine göre kontrol bloğundaki mantıksal işlemler işlemcinin nasıl çalışacağını hangi alanların aktif hangilerini pasif olacağını kararlaştırır.

Sonraki 9 bit ise kaydedici adreslerini belirtmek için kullanılır. ADD (toplama) komutu üzerinden örnek verirsek; KY1 deki son veriyi KY2 deki son veri ile toplayıp çıkan sonucu HY'ye yazar. Bu sırada 3 bit ile HY, 3 bit ile KY1 ve 3 bit ile KY2 adreslenir. Her bir kaydedici adresi için 2^3 farklı değer verilebilir. Bu sayede toplamda 8 kaydediciye kadar adresleme yapılabilir.

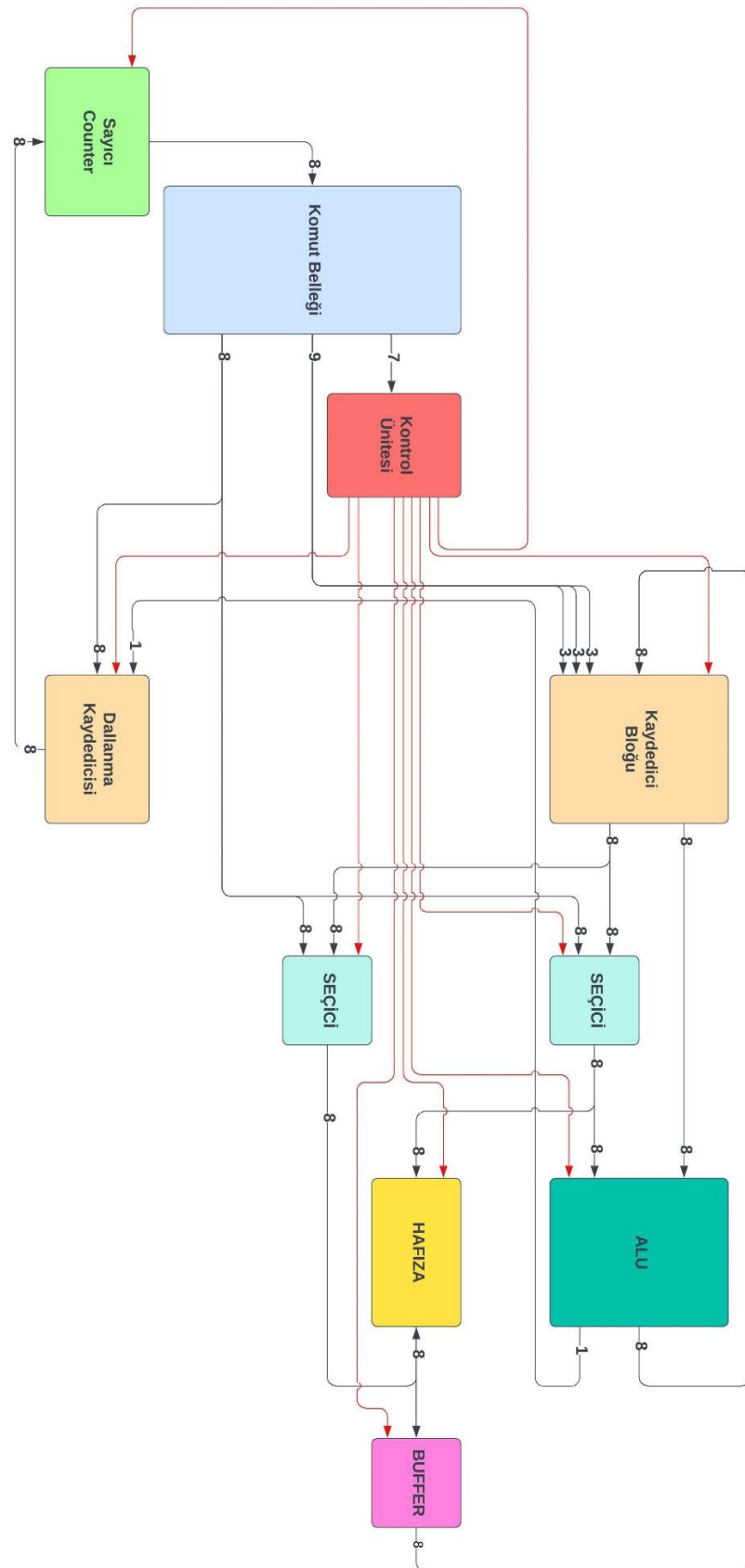
Sonraki 8 bit ise çok amaçlı kullanabilen 8 bitlik veri içeren alandır. Tablo 1'de gösterilen komutların arasında anlık değer kullanan komutlar bulunmaktadır. Anlık değer kaydedici veya hafıza birimlerine kaydedilecek bir değeri barındırabildiği gibi bir hafıza adresini de barındırabilir. Ayrıca dallanma komutlarında da programın ne kadar komut atlayacağını belirtmek için kullanılır.



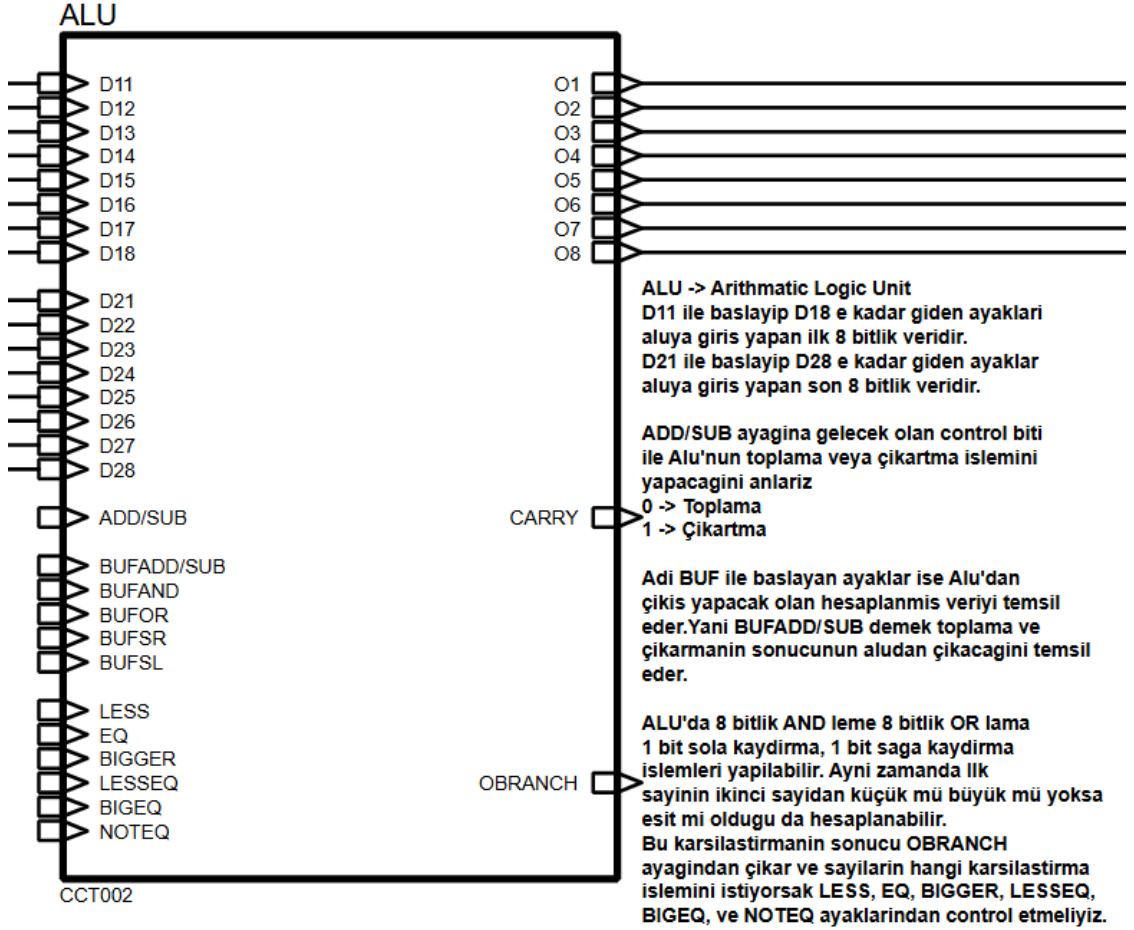
Şekil 1 - Komut İşleme Mantığı

(HY-> Hedef kaydedici/yazmaç, KY1-> Kaynak kaydedici/yazmaç 1, KY2-> Kaynak kaydedici/yazmaç 2)

Şekil 2 Mimari



Şekil 2’de tasarlanan mimarinin bir şemasını görebilirsiniz. Komut belleğinden gelen 24 bit ile komutlar işlenir. Önceden de bahsedildiği üzere 7 bit ile kontrol ünitesi işlemcinin birçok alanını kontrol eder. Kırmızı renk ile gösterilen oklar kontrol ünitesi tarafından kontrol edilen alanları temsil etmektedir. Tek bir ok ile temsil edilmesine rağmen çok sayıda bit ile kontrolleri bulunmaktadır. Örnek olması açısından Şekil 4’ü inceleyebilirsiniz.

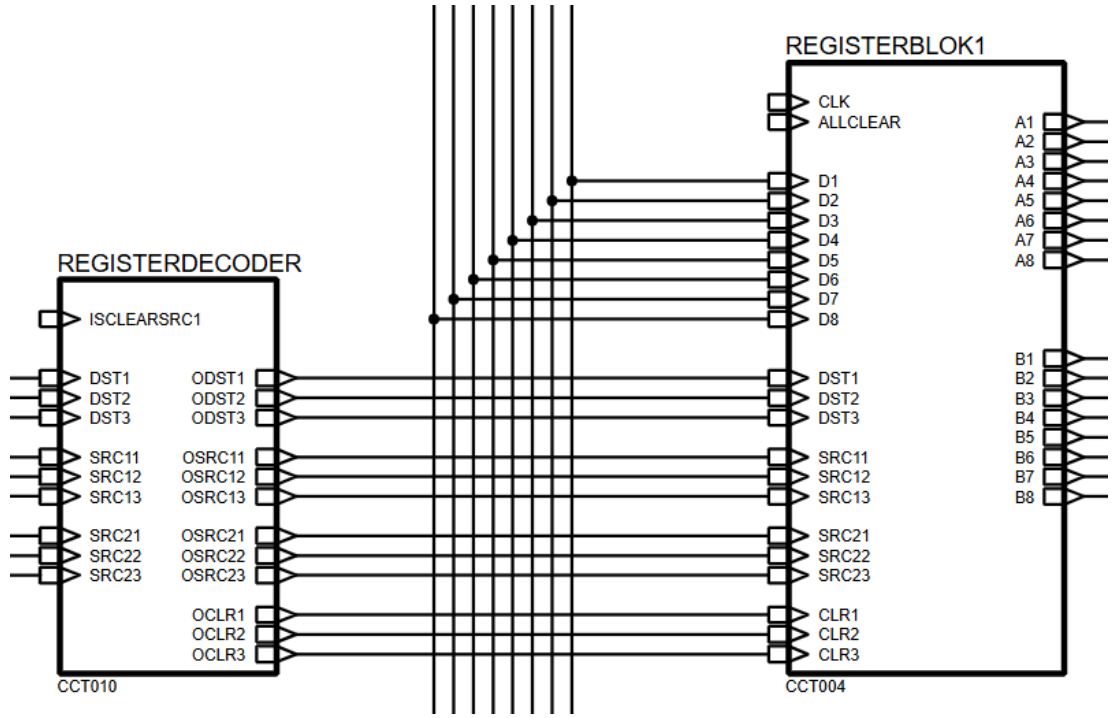


Şekil 4 Proteus Üzerinde ALU'nun ana devre görünüşü

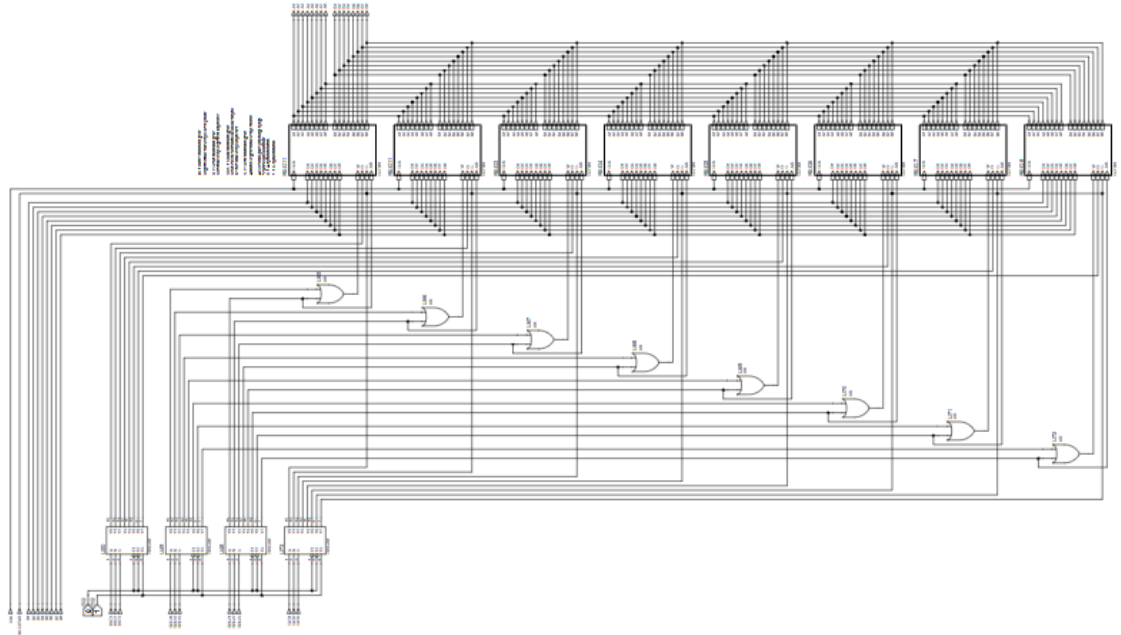
Komut işleme mantığı başlığı altında bahsedilen kaydedici adreslemenin yapılmasını sağlayan 9 bitlik kısım kaydedici blokuna bağlıdır. Kaydedicideki bir değeri silme komutu geldiyse silme işlemini yapan ayaklara, kaydediciye bir veri kaydetme komutu geldiyse normal adresleme ayaklarına adres değerini ulaştıran bir ara seçici vardır. Bu ara seçiciden çıkan adresler doğrudan kaydedici blokuna ulaşır. Kaydedici bloku adres çözümleyiciyi içerisinde barındırır. (Bkz. Şekil 5)

2.4 Kaydedici / Kaydedici Bloğu Tasarlanması

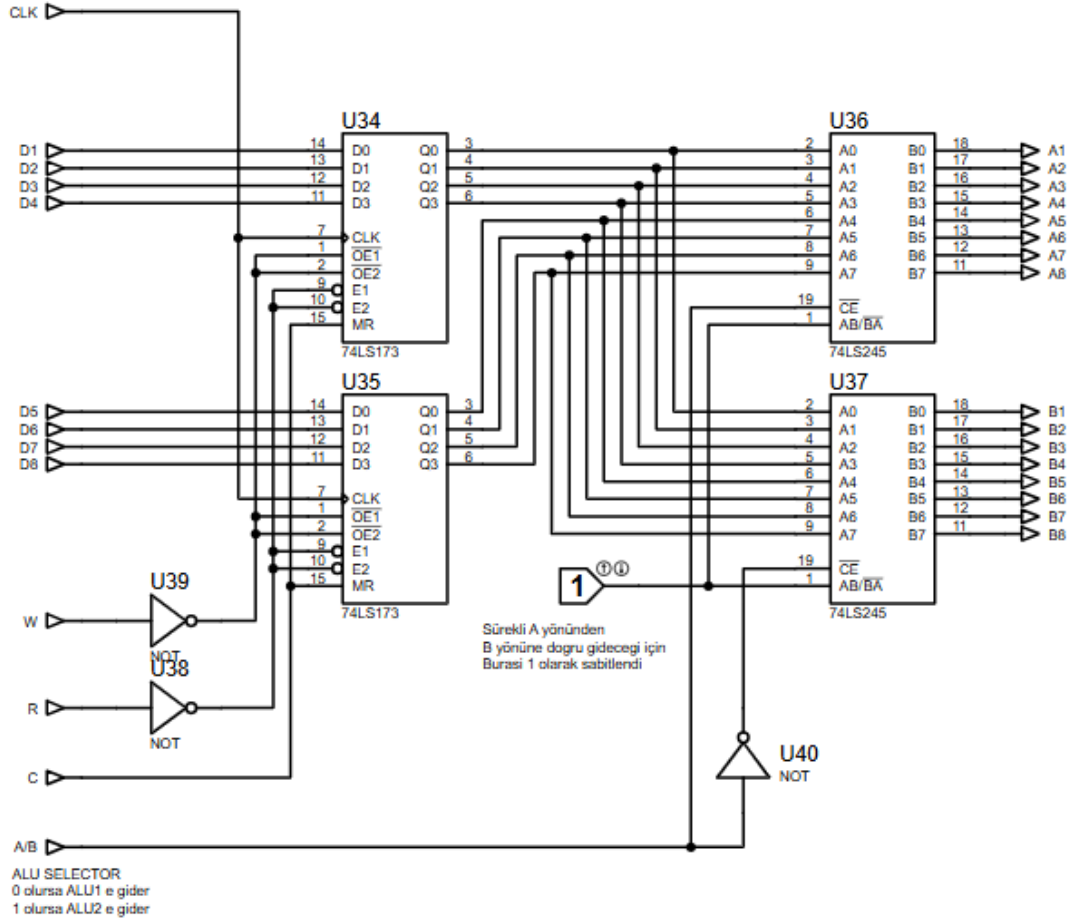
Ana görünümü Şekil 6’da görüleceği üzere 8 adet kaydediciden oluşmuştur. Tek bir kaydedici için 2 adet 4 bitlik 74LS173 entegre devresi ve çıkış yapacağı ayağı seçmek için 2 adet 8 bitlik 74LS245 entegre devresi (buffer) kullanılmıştır (Bkz. Şekil 7).



Şekil 5 Kaydedici bloğu (Solda ara kontrol bloğu olan REGISTERDECODER bulunuyor. Program belleğinden gelen kaydedici adresleri REGISTERDECODER'a ulaşıyor. REGISTERDECODER devresinden çıkış yapan adresler REGISTERBLOK1 devresinin ilgili bölümlerine bağlanıyor. Resmi ortadan iki ayırtırmış gibi görünen 8 adet bağlantılar 8 bitlik veri yolunu temsil eder. REGISTERBLOK1 devresinin iç yapısını Şekil 5'ten inceleyebilirsiniz. REGISTERBLOK1 den çıkan bağlantılar daha sonra işlemcinin ilgili bölümlerine ulaşıyor. Tüm devrenin gösterildiği resim üzerinden gösterilecektir.)



Şekil 6 Kaydedici Bloğunun İçi (Child Circuit) Üstte 8 adet kaydedici, her biri Şekil 7 de görülecek şekildedir. Her birinden 8 bit çıkar ancak ALU'nun ilk 8 bit kısmına mı yoksa ikinci 8 bit kısmına mı gideceğini seçmek için adresle çözümleyiciler çalışır. Altta 4 adet adres çözümleyici bulunur. Bunlar sırasıyla hedef kaydedici/yazmaç, kaynak kaydedici/yazmaç 1, kaynak kaydedici/yazmaç 2, silinecek kaydedici çözümleyicileridir.



Şekil 7 Tek bir kaydedici

Tasarlanan komut setindeki tüm komutlar kaydedicileri kullanmıyorlar. Bazı komutlar ise sadece hedef veya sadece kaynak kaydedicileri kullanıyorlar. Tablo 2’de yukarıdaki açıklamanın bir sonucunu görebilirsiniz.

		7 BİT	3BİT	3 BİT	3 BİT	8 BİT	
	Komut	Kontrol Ünitesi Kodu	HY	KY1	KY2	Anlık Değer	Açıklama
1	ADD	0000000	✓	✓	✓	X	HY = KY1 + KY2
2	ADDIMM	0000001	✓	✓	X	✓	HY = KY1 + ANLIK
3	SUB	0000010	✓	✓	✓	X	HY = KY1 – KY2
4	SUBIMM	0000011	✓	✓	X	✓	HY = KY1 – ANLIK
5	AND	0000100	✓	✓	✓	X	HY = KY1 && KY2
6	ANDIMM	0000101	✓	✓	X	✓	HY = KY1 && ANLIK
7	OR	0000110	✓	✓	✓	X	HY = KY1 KY2
8	ORIMM	0000111	✓	✓	X	✓	HY = KY1 ANLIK
9	SHIFLEFT	0001000	✓	✓	X	X	HY = KY1[0:6]
10	SHIFRIGHT	0001001	✓	✓	X	X	HY = KY1[1:7]
11	IMMTOREG	0001010	✓	X	X	✓	HY = ANLIK
12	MEMTOREG	0001011	✓	X	X	✓	HY = MEM(ANLIK)
13	REGTOREG	0001100	✓	X	✓	X	HY = KY2
14	REGTOMEM	0001101	X	✓	X	✓	MEM(ANLIK) = KY2
15	BEQ	0001110	X	✓	✓	✓	IF KY1 == KY2 JUMP ANLIK
16	BNE	0001111	X	✓	✓	✓	IF KY1 != KY2 JUMP ANLIK

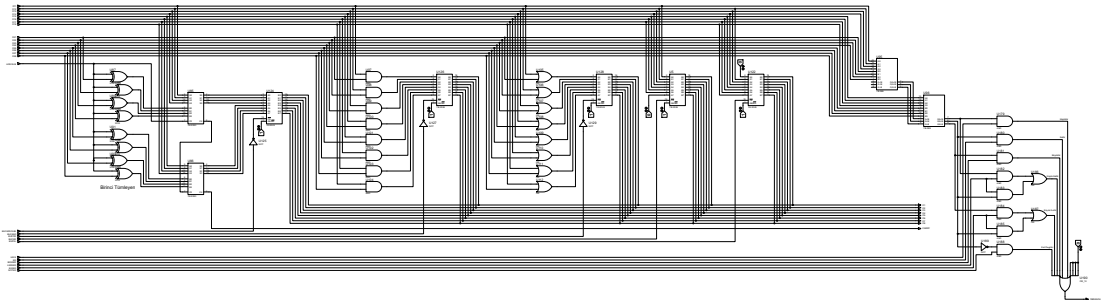
17	BLT	0010000	X	✓	✓	✓	IF KY1 < KY2 JUMP ANLIK
18	BGT	0010001	X	✓	✓	✓	IF KY1 > KY2 JUMP ANLIK
19	BGE	0010010	X	✓	✓	✓	IF KY1 >= KY2 JUMP ANLIK
20	BLE	0010100	X	✓	✓	✓	IF KY1 <= KY2 JUMP ANLIK
21	PRINTREG	0010100	X	X	✓	X	OUTPUT(KY2)
22	IMMTOPRINTMEM	0010101	X	X	X	✓	OUTPUT(MEM(ANLIK))
23	WRITEREG	0010110	✓	X	X	X	HY = INPUT
24	IMMTOWRITEMEM	0010111	X	X	X	✓	MEM(ANLIK) = INPUT
25	CLEARALLREG	0011000	X	X	X	X	ALLREG = 0
26	CLEARSPECREG	0011001	X	✓	X	X	REG = 0

Tablo 2

Komutlar ve kullandıkları kaydedici adresleri

2.5 Aritmetik Mantık Birimi Tasarlanması

Aritmetik Mantık birimi işlemcinin beyni niteliğindedir. Aritmetiksel ve mantıksal işleri yapabilir. Bu projede toplama çıkarma aritmetik işlemleri ve AND, OR gibi mantıksal ve giriş seviyesi işlemleri yapabilecek bir ALU tasarlanmıştır. Ayrıca bit işlemlerinden birer bit sola ve sağa kaydırma komutları ve bunu sağlayacak devreler ve karşılaştırma komutları ve bunu sağlayacak devreler mevcuttur. Proje önerisinde sunulan ifade yenilenmiştir. Öneride söylenen “sadece kaydedici 3 üzerinden kaydırma işlemleri yapılabilir” ifadesi daha kapsamlı kaydedici bloğu ve yeni kontrol birimleri tasarlanarak değiştirilmiş oldu. Son haliyle herhangi bir kaydedici üzerinden kaydırma işlemleri yapılabilir.



Şekil 8 ALU iç devresi (child circuit) Şekil 4 de gösterilen ana devrenin iç görünümüdür. Soldan sağa sırasıyla toplama ve çıkarma devresi, AND ile toplama devresi, OR ile toplama devresi, sola kaydırma devresi, sağa kaydırma devresi ve karşılaştırma işlemlerini gerçekleştiren karşılaştırma devresi

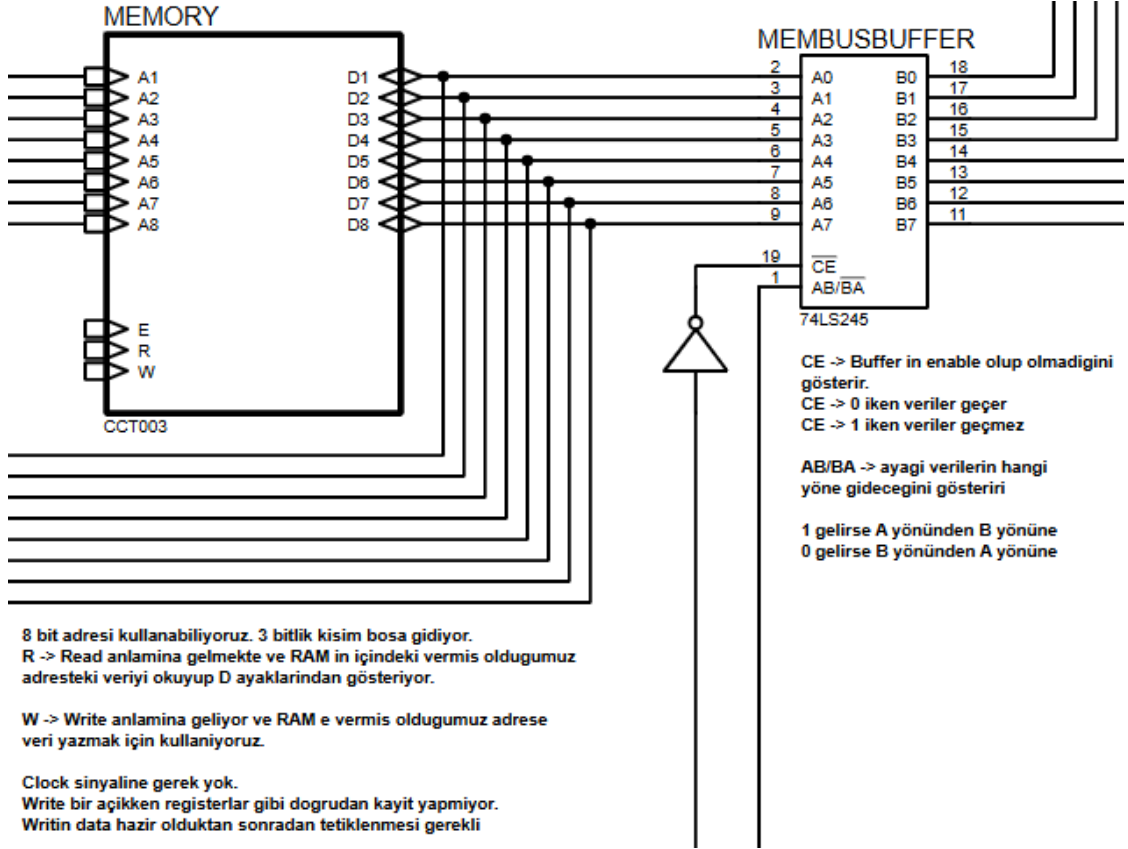
2.6 Hafıza Birimi Tasarlanması

Hafıza birimi için 11 bit adreslenebilen ve her adresinde 8 bit veri saklayabilen statik ram kullanılmıştır. Piyasada 6116 olarak adı geçmektedir ve 16 kilobayttır. Okuma ve yazma işlemleri aynı ayakları üzerinden olur. Bunu kontrol edebilmek için buffer ve seçici gibi elemanlar kullanılarak kontrol alanları eklenmiştir. Şekil 9 da BUS ile Hafıza biriminin arasındaki veri çakışmasını önlemesi amacıyla bir buffer konulmuştur.

Komutlardan da görülebileceği gibi Hafıza birimi adreslenirken kaydedicilerden gelen veri veya anlık veri kullanılabilir. Yani komutlar bir

derleyici ile oluştururken Hafıza işlemlerini kontrol edebilmesi için o komut için anlık veri oluşturabilir veya bir veya birden çok kaydediciyi adresleme yapmak için kullanabilir.

Veri ise yine aynı şekilde kaydedicilerden veya anlık veriden gelebilir. Ayrıca şu an ilgili komutu bulunmasa bile ileride giriş çıkış ünitesinden gelen veri doğruca hafıza birimine yazılabilir. Bunu sağlayan kontrol alanları mevcuttur.

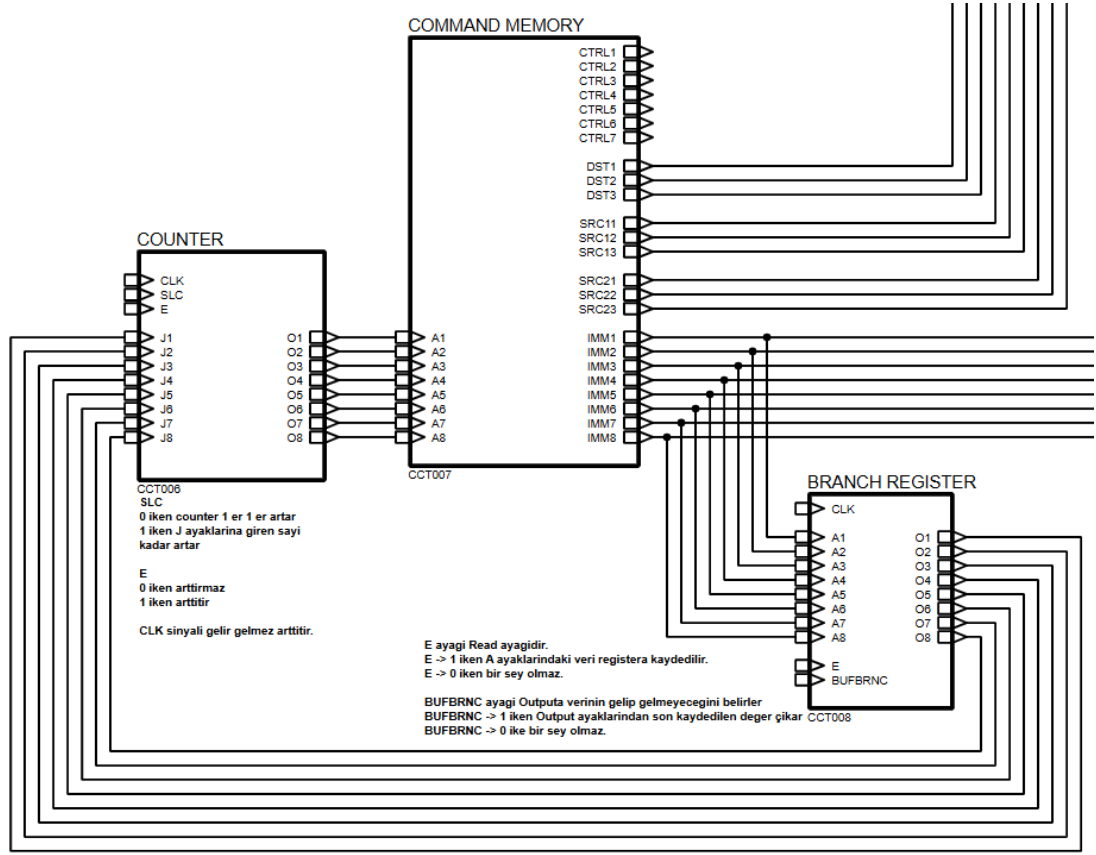


Şekil 9 Hafıza birimi

2.7 Sayıcı (Counter) Tasarlanması

Sayıcı program belleğindeki komutları tek tek geçmeye yarayan devredir. İşlemci her bir döngüsünde bir komut tamamlayacak şekilde tasarlanmıştır ve RISC-V komut setinin aksine tek tip uzunlukta komutlar içerir. Bu nedenle sayıcı tasarımı kısmen kolaydır ve basittir.

Bu projede dallanma komutları da işlenebilsin diye sayıcı ile dallanma kaydedicisi birbirine bağlıdır ve kontrol alanları vardır. ALU yazısından bahsedilen karşılaştırma işlemlerini yapan kısımdan çıkan sonuç doğrudan dallanma kaydedicisine ve sayıcıya bağlıdır. Buradan gelen sonuca göre sayıcı dallanma kaydedicisindeki adım kadar atlayabilir veya 1'er 1'er artmaya devam edebilir.



Şekil 10 Sayıcı ve Dallanma Kaydedicisi

2.8 Kontrol Ünitesi Tasarımı

Yukarıdaki resimlerin bir kısmından görebileceğiniz gibi işlemci üzerindeki devrelerin her biri en azından 1 kontrol alanı içeriyor. Örneğin buffer ve seçiciler verinin hangi yol üzerin geçeceğine karar vermemize yardımcı olan devreler. Bu devrelere gelen 1 veya 0 bit verisi ile işlemcinin ürettiği veya kaydettiği sonuç tamamen değişmektedir. Dolayısıyla her bir komut için her bir kontrol alanını tek tek kontrol etmek ve her birinde nasıl bir davranış sergilemesi gerektiğinin çözümlenmesi gereklidir.

Bu açık ara işlemci üzerindeki en karmaşık devredir ve oluşturulurken Karnough haritası tekniği kullanılır.

İşlemci üzerinde kontrol alanı içeren 11 tasarlanan devre vardır ve bunların toplam 28 adet kontrol alanı vardır. Kontrol alanlarının çoğu aktif veya aktif değil (enable) kontrolleridir. Her bir devrenin o anda aktif olup olmadığını ve bir hesaplama üretip üretmeyeceğini veya kayıt yapıp yapmayacağını belirtir. Bazı devrelerde ise aktiflik kontrolü yoktur çünkü onlar her komutta çalışan devrelerdir (örn: buffer, bazı seçiciler).

Karnough Map			Counter	Branch Register		IMMTO BUS BUFFER	REGOR IMM1	REG_OR_IMM2		Memory			MEM_BUS BUFFER		REGISTER BLOK	Alu												REGISTER DECODER	Input Output	
			SLC	E	BUF BRNC	E	SLC	SLC	E-MEM D	E	R	W	E	IN/OUT	CLEAR	ADD/SUB	BUFF ADD/SUB	BUF AND	BUF OR	BUF SR	BUF SL	LESS	EQ	BIG GER	LESS EQ	BIG EQ	NOT EQ	ISCLEAR SRC1	ENABLE IN	LCD/D
	Komut	Kontrol Ünitesi Kodu																												
1	ADD	0000000	0	0	0	1	0	X	X	0	X	X	0	X	0	0	1	0	0	0	0	0	0	0	0	0	0	0	X	0
2	ADDIMM	0000001	0	0	0	1	1	X	X	0	X	X	0	X	0	0	1	0	0	0	0	0	0	0	0	0	0	0	X	0
3	SUB	0000010	0	0	0	1	0	X	X	0	X	X	0	X	0	1	1	0	0	0	0	0	0	0	0	0	0	0	X	0
4	SUBIMM	0000011	0	0	0	1	1	X	X	0	X	X	0	X	0	1	1	0	0	0	0	0	0	0	0	0	0	0	X	0
5	AND	0000100	0	0	0	1	0	X	X	0	X	X	0	X	0	X	0	1	0	0	0	0	0	0	0	0	0	0	X	0
6	ANDIMM	0000101	0	0	0	1	1	X	X	0	X	X	0	X	0	X	0	1	0	0	0	0	0	0	0	0	0	0	X	0
7	OR	0000110	0	0	0	1	0	X	X	0	X	X	0	X	0	X	0	0	1	0	0	0	0	0	0	0	0	0	X	0
8	ORIMM	0000111	0	0	0	1	1	X	X	0	X	X	0	X	0	X	0	0	1	0	0	0	0	0	0	0	0	0	X	0
9	SHIFLEFT	0001000	0	0	0	1	0	X	X	0	X	X	0	X	0	X	0	0	0	0	0	1	0	0	0	0	0	0	X	0
10	SHIFTRIGHT	0001001	0	0	0	1	0	X	X	0	X	X	0	X	0	X	0	0	0	1	0	0	0	0	0	0	0	0	X	0
11	IMMTOREG	0001010	0	0	0	0	X	X	X	0	X	X	0	X	0	X	0	0	0	0	0	0	0	0	0	0	0	0	X	0
12	MEMTOREG	0001011	0	0	0	1	1	X	0	1	1	0	1	1	0	X	0	0	0	0	0	0	0	0	0	0	0	0	X	0
13	REGTOREG	0001100	0	0	0	1	X	0	1	0	X	X	1	1	0	X	0	0	0	0	0	0	0	0	0	0	0	0	X	0
14	REGTOMEM	0001101	0	0	0	1	1	0	1	1	0	1	1	0	0	X	0	0	0	0	0	0	0	0	0	0	0	0	X	0
15	BEQ	0001110	1	1	1	1	0	X	X	0	X	X	0	X	0	X	0	0	0	0	0	0	1	0	0	0	0	0	X	0
16	BNE	0001111	1	1	1	1	0	X	X	0	X	X	0	X	0	X	0	0	0	0	0	0	0	0	0	0	1	0	X	0
17	BLT	0010000	1	1	1	1	0	X	X	0	X	X	0	X	0	X	0	0	0	0	0	1	0	0	0	0	0	0	X	0
18	BGT	0010001	1	1	1	1	0	X	X	0	X	X	0	X	0	X	0	0	0	0	0	0	0	1	0	0	0	0	X	0
19	BGE	0010010	1	1	1	1	0	X	X	0	X	X	0	X	0	X	0	0	0	0	0	0	0	0	0	1	0	0	X	0
20	BLE	0010011	1	1	1	1	0	X	X	0	X	X	0	X	0	X	0	0	0	0	0	0	0	0	1	0	0	0	X	0
21	PRINTREG	0010100	0	0	0	1	X	0	1	0	X	X	1	1	0	X	0	0	0	0	0	0	0	0	0	0	0	0	0	1
22	IMMTOPRINTMEM	0010101	0	0	0	1	1	X	0	1	1	0	1	1	0	X	0	0	0	0	0	0	0	0	0	0	0	0	0	1
23	WRITEREG	0010110	0	0	0	1	X	X	X	0	X	X	0	X	0	X	0	0	0	0	0	0	0	0	0	0	0	0	1	0
24	IMMTOWRITEMEM	0010111	0	0	0	1	1	X	0	1	0	1	1	0	0	X	0	0	0	0	0	0	0	0	0	0	0	0	1	0
25	CLEARALLREG	0011000	0	0	0	1	X	X	X	0	X	X	0	X	1	X	0	0	0	0	0	0	0	0	0	0	0	0	x	0
26	CLEARSPECREG	0011001	0	0	0	1	X	X	X	0	X	X	0	X	0	X	0	0	0	0	0	0	0	0	0	0	0	1	x	0

Tablo 3

Karnough Map

File Edit View Minimize Preferences Help

Sum of Products (DNF) Minimize All

Project Explorer

Variables

Bit Order: MSB first

Count: 5

1. x1

2. x2

3. x3

4. x4

5. x5

Functions

Count: 28

1. counterslc

2. branchE

3. branchBuf

4. immtobusE

5. regorimm1Slc

6. regorimm2Slc

7. regorimm2E

8. memoryE

9. memoryR

10. memoryW

11. membufferE

12. memBufferInOut

13. RegisterClear

14. aluAddSub

15. aluAddSubBuffer

16. aluAndBuf

17. aluOrBuf

18. aluSrBuf

19. aluSrBuf

20. aluLess

21. aluEq

22. aluBigger

23. aluLessEq

24. aluBigEq

25. aluNotEq

Table View

#	x1	x2	x3	x4	x5	counterslc	branchE	branchBuf	immtobusE	regorimm1Slc	regorimm2Slc	regorimm2E	memoryE	memoryR	memoryW	membufferE	memBufferInOut	RegisterClear	aluAddSub	aluAddSubBuffer
0	0	0	0	0	0	0	0	0	1	0	X	X	0	X	X	0	X	0	X	1
1	0	0	0	0	1	0	0	0	1	1	X	X	0	X	X	0	X	0	X	1
2	0	0	0	1	0	0	0	0	1	0	X	X	0	X	X	0	X	0	1	1
3	0	0	0	1	1	0	0	0	1	1	X	X	0	X	X	0	X	0	1	1
4	0	0	1	0	0	0	0	0	1	0	X	X	0	X	X	0	X	0	X	0
5	0	0	1	0	1	0	0	0	1	1	X	X	0	X	X	0	X	0	X	0
6	0	0	1	1	0	0	0	0	1	0	X	X	0	X	X	0	X	0	X	0
7	0	0	1	1	1	0	0	0	1	1	X	X	0	X	X	0	X	0	X	0
8	0	1	0	0	0	0	0	0	1	0	X	X	0	X	X	0	X	0	X	0
9	0	1	0	0	1	0	0	0	1	0	X	X	0	X	X	0	X	0	X	0
10	0	1	0	1	0	0	0	0	0	X	X	X	0	X	X	0	X	0	X	0
11	0	1	0	1	1	0	0	0	1	1	X	0	1	1	0	1	1	0	X	0
12	0	1	1	0	0	0	0	0	1	X	0	1	0	X	X	1	1	0	X	0
13	0	1	1	0	1	0	0	0	1	1	0	1	1	0	1	1	0	0	X	0
14	0	1	1	1	0	1	1	1	1	0	X	X	0	X	X	0	X	0	X	0
15	0	1	1	1	1	1	1	1	1	0	X	X	0	X	X	0	X	0	X	0
16	1	0	0	0	0	1	1	1	1	0	X	X	0	X	X	0	X	0	X	0
17	1	0	0	0	1	1	1	1	1	0	X	X	0	X	X	0	X	0	X	0
18	1	0	0	1	0	1	1	1	1	0	X	X	0	X	X	0	X	0	X	0
19	1	0	0	1	1	1	1	1	1	0	X	X	0	X	X	0	X	0	X	0
20	1	0	1	0	0	0	0	0	1	X	0	1	0	X	X	1	1	0	X	0
21	1	0	1	0	1	0	0	0	1	1	X	0	1	1	0	1	1	0	X	0
22	1	0	1	1	0	0	0	0	1	X	X	X	0	X	X	0	X	0	X	0

Map View

c b b i r r r m m m m m R a a a a a a a a a a a a r i i

x1

x2

x3

x4

x5

Properties

Table View

Colors

Functions

Colors

Grid

Visible: ☒

Horizontal: ☒

Vertical: ☒

Contain Labels: ☐

Color:

Labels

Rotation: Counte

Numbers

Visible: ☒

Label: #

Colors

Variables

Visible: ☒

Colors

Minimization Results

Expression validated.

Factor Out Variables: ☒

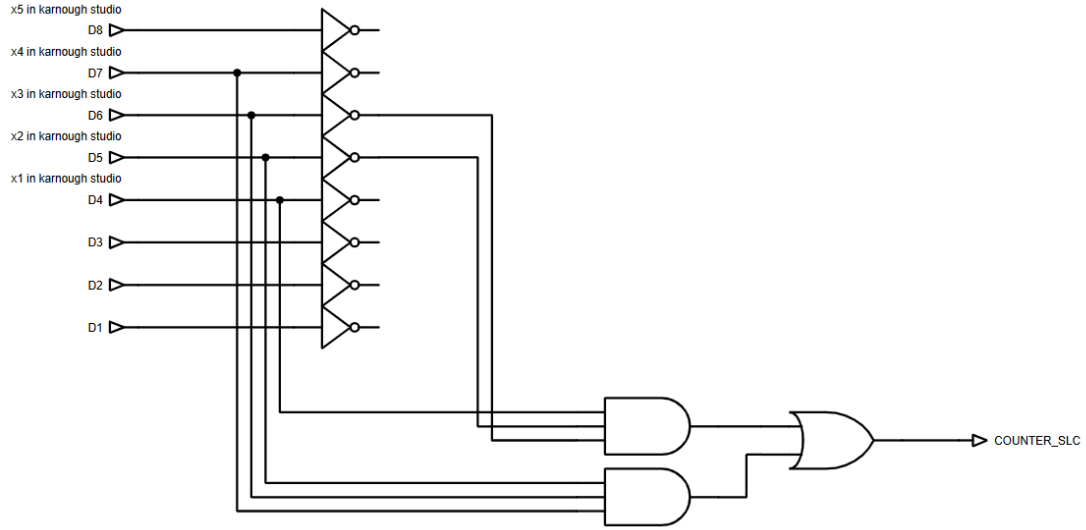
branchE = (x1 · x2 · x3) + (x2 · x3 · x4)

7 operation(s).

Copy to Clipboard

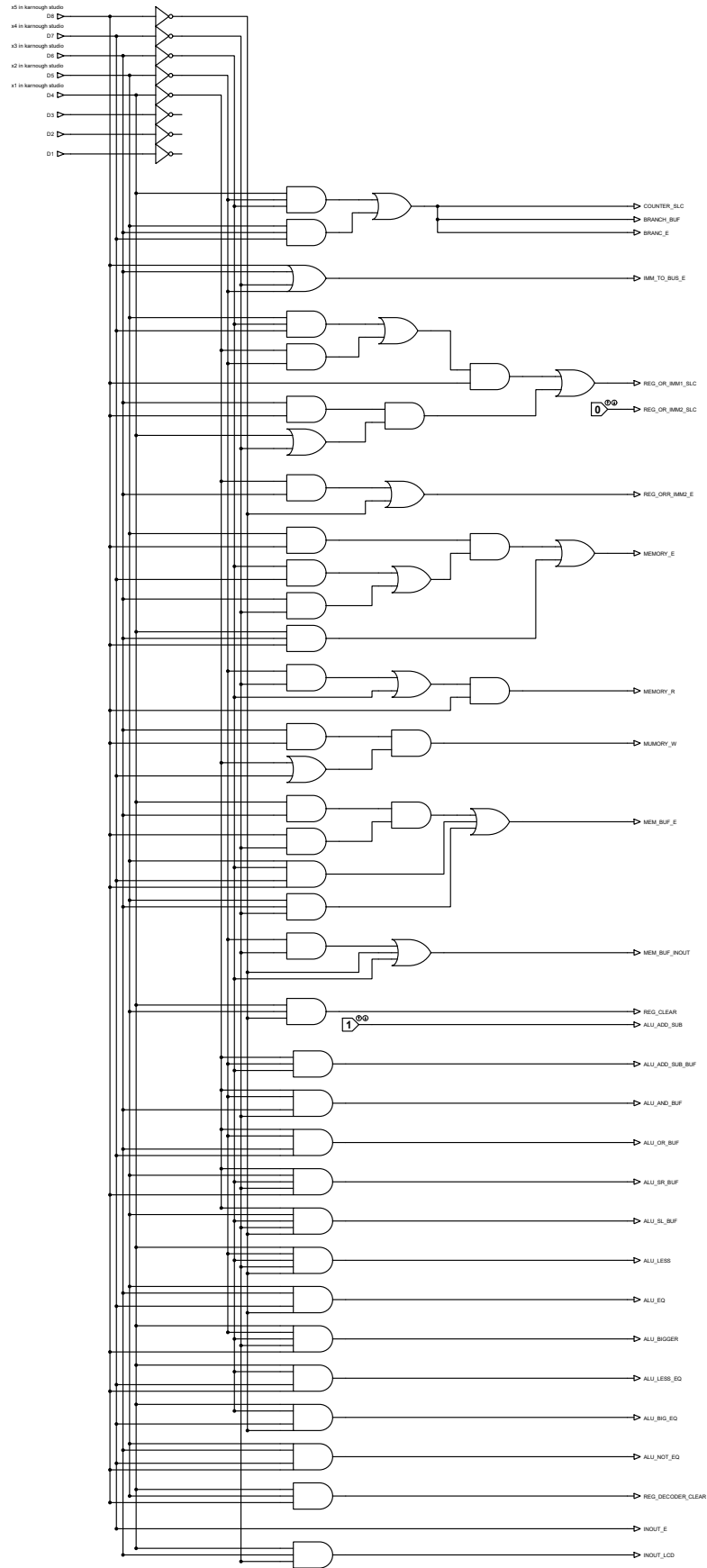
Export...

Karnough haritası oluştururken Şekil 11 ‘den görebileceğiniz üzere Karnough Studio adında program kullanılmıştır. Sağ altta ilgili kontrol alanını nasıl kontrol edebileceğimiz Sum Of Product (DNF) formatında gösterilmektedir. Buradaki nokta işaretleri AND , artı işaretleri OR ve dik çizgi işaretleri de o değerin tersini ifade etmektedir. Bu bilgiler doğrultusunda 28 kontrol alanı için de ayrı ayrı AND ve OR kapıları kullanarak komut koduna göre bir kontrol ünitesi tasarlanmıştır. Örnek olması için sadece counterSLC kontrol alanının hesaplanması şekil 12’deki gibi yapılacaktır.



Şekil 12 Counter SLC Kontrol alanı için devre $((x1 \cdot x2 \cdot x3) + (x2 \cdot x3 \cdot x4))$

Tüm kontrol ünitesini Şekil 13’teki görselden görebilirsiniz. Kontrol ünitesinin Şekil 3’teki mimariye eklenmiş halini de Şekil 14’de görebilirsiniz.



Şekil 13 Kontrol Ünitesi İçi (Child circuit)

3. Sonuç:

3.1 Harward Mimarisinin Götürüleri

Harward mimarisinin benimsenmesi ile program belleği ile veri belleğinin ayrı olması veriye erişim anlamında kolaylık getirirse de kontrol ünitesindeki karmaşıklığın artması ve yer maliyetini artması gibi sonuçlar doğurmuştur.

Tek komutta tek işlem mantığının belirlenmesi de aynı şekilde kontrol ünitesinin karmaşıklığını arttırmıştır.

Yukarıda sayılan iki nedenden dolayı elde edilen üstünlüğün kontrol ünitesi tasarlanması sırasında kaybedilmiş gibi görünüyor. Bu cümlelerin sonucuna karar verecek olan şey işlemcinin kullanım alanı ve işlemleri olacaktır.

8 bitlik ve 1 kilobaytlık bir işlemci tasarlamış olmamız günümüzde çok az konumda ve çok az bir işlem yapabilecek şekilde olduğundan şimdilik işe yarar bir yönü yoktur.

3.2 Derleyici Tasarım Eksikliği

Proje önerisinde planlanan ancak sonrasında çeşitli nedenlerden dolayı proje sonucuna yetiştirilemeyen derleyici tasarımı konusunun eksik olması nedeniyle işlemci üzerinde çok sayıda ve hızlı bir şekilde testler yapılamamaktadır. Maalesef şimdilik sadece program belleğine manuel olarak kayıtların atılmasının ardından işlemcinin çalışmasını adım adım izleyebilmekteyiz. Elde edilen sonuçlar doğrudan görüntülenemediği için simülasyon ortamında tek tek incelenip işlemcinin doğru çalışıp çalışmadığına karar vermekteyiz.

3.3 Başarılarımız

Komut uzunluğunu tek tür ve 24 bit uzunluğunda yaptığımızdan dolayı her bir saat döngüsünde tek bir işlem yapabilen işlemci tasarlanmış oldu. 128 farklı komut oluşturulabilecek kadar komut çeşidi havuzu vardır. Şimdilik sadece 26 komut kullanılmıştır. İşlemciyi geliştirmek isteyen bir başkası için oyun alanı bırakmış olduk.

Var olan komutları ve mimariyi değiştirmedeği sürece eklediği her bir devre ve komut ile işlemciyi bir sonraki seviyeye yükseltebilir.

8 bit olarak tasarlanmasına rağmen aynı kontrol üniteleri ve işlemler bırakılarak sadece ilgili alanlardaki elektronik devreler değiştirilerek 16, 32 veya 64 olacak şekilde güncellenebilir.

4. ıktılar (Yayınlar,sunumlar v.b.)

Herhangi bir yayın veya sunum yapılmamıştır. Bildiri yazısı yazılmıştır ancak yayınlanmamıştır.

5. Proje ile ilgili harcama kalemleri hakkında ayrıntılı bilgi

Herhangi bir harcama kalemi bulunmamaktadır. Proje simülasyon ortamında üretilmiştir. PCB devre haline getirilip üretiminin planlanması üzerinde düşünülebilir.

PROJE YÜRÜTÜCÜSÜNÜN ADI – SOYADI - İMZA	DANIŞMANIN ADI – SOYADI - İMZA
Musa Akyüz	Dr. Öğr. Üyesi Ümit Şentürk
	

Tarih : 18.09.2024